PYTHON FOR DATA ENGINEERS

Prepared By : Kaviraj T U

INCREMENTAL LOADS IN PYTHON

(production-grade design)

Prepared By : Kaviraj T U

Why Incremental Loads Matter?

- In real-world data engineering, data keeps growing every day.
- Running a **full load** repeatedly:
  ➜ Wastes compute resources
  ➜ Increases job time
  ➜ Causes duplication risks
  ➜ Doesn't scale for large datasets

- This is why production pipelines use **incremental loads**

1. What is an Incremental Load?

- An incremental load only processes **new or changed data** since the last pipeline run.

- Instead of loading everything:
  ➜ Full Load:
  Process all records every time

- We do:
  ➜ Incremental Load:
  Process only new or updated records

- **Real world example:**
  ➜ Orders Table (Source System)

order_id	created_at
101	2025-02-01
102	2025-02-02
103	2025-02-03

- Pipeline runs on **Feb 4**.
- Instead of loading all records, we load only:

  → WHERE created_at > last_run_time

2. Full Load vs Incremental Load

Aspect	Full Load	Incremental Load
Data processed	Entire dataset	Only new/changed data
Speed	Slow for large tables	Fast
Cost	High	Lower
Complexity	Simple	Slightly complex
Production use	Rare	Standard practice

3. Types of Incremental Loads

i) Timestamp-Based Incremental Load:

- Most common approach.
- Load records where:

  → created_at > last_run_time
- Used when:

  → Table has timestamps

  → Data is append-only

ii) ID-Based Incremental Load

➜ WHERE order_id > last_max_id

- Used when:
  ➜ IDs are sequential
  ➜ No reliable timestamps

iii) Change Data Capture (CDC)

- Tracks:
  ➜ Inserts
  ➜ Updates
  ➜ Deletes

- More advanced, real-time approach.

4. Core Components of an Incremental Pipeline

- Every incremental pipeline needs:

  ➜ Watermark / Last Run Time

  ➜ Filter Logic

  ➜ State Storage

  ➜ Load Mechanism

What is a Watermark?

- A **watermark** is the last processed value.

- Example:
  → **last_run_time = 2025-02-03 23:59:59**

- Next run loads:
  → **records where created_at > last_run_time**

5. Simple Python Incremental Load Example

- **Step 1**: Store Last Run Time

  → last_run_time = "2025-02-03"

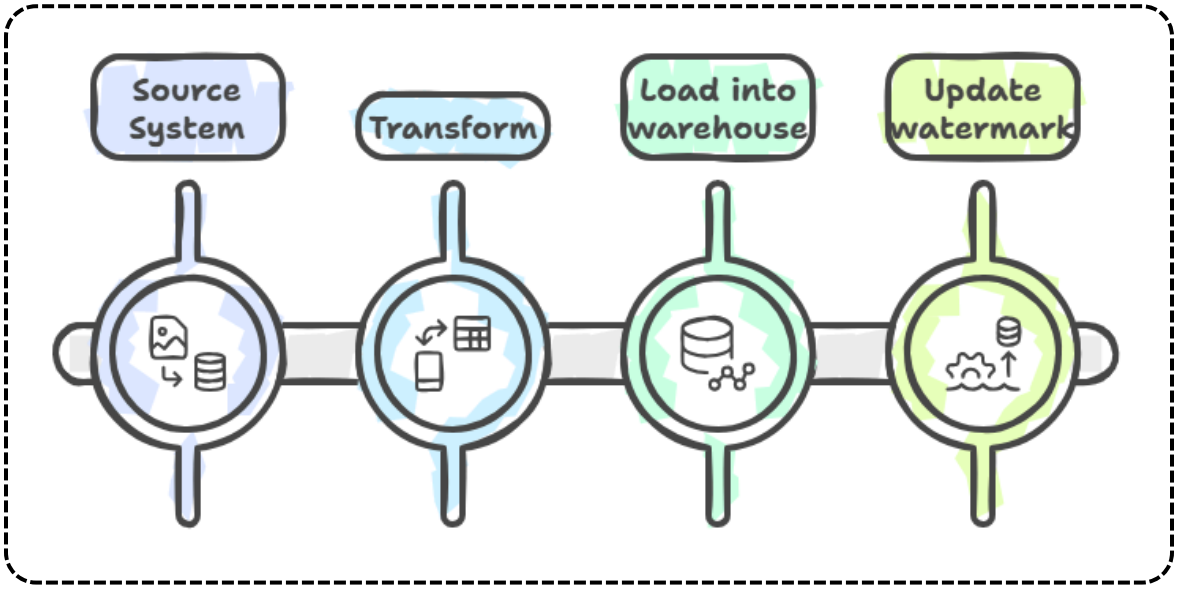
- **Step 2**: Filter New Records

```
def incremental_filter(records, last_run_time):
    new_records = []
    for record in records:
        if record["created_at"] > last_run_time:
            new_records.append(record)
    return new_records
```

- **Step 3:** Process Only New Data

  ➜ new_data = incremental_filter(source_data, last_run_time)

6. Real Pipeline Flow



7. Handling Updates in Incremental Loads

- Append-only logic fails when:

  ➜ Records get updated
  ➜ Old data changes

- **Example:**

order_id	amount	updated_at
101	250	2025-02-01
101	300	2025-02-04

- Solution:

  ➜ **Use updated_at column instead of created_at**

8. Late-Arriving Data Problem

- **Scenario:**

  ➜ Record created on **Feb 2**
  ➜ Arrives on **Feb 5**

- If watermark is **Feb 4:**
  ➜ Record is missed

- **Solution: Overlapping Window**

  ➜ Instead of:

  created_at > last_run_time

  ➜ Use:

  created_at > last_run_time - 1 day

- Then deduplicate downstream.

9. Late-Arriving Data Problem

Storage	Example
Metadata table	last_run_time column
Config file	JSON/YAML
Workflow tool	Airflow variables
Database	Control table

10. Common Mistakes

- Using local time instead of UTC
- Not handling updates
- Ignoring late-arriving data
- Hardcoding watermarks
- No deduplication logic

11. Production Best Practices

- Always use timestamps when available
- Use UTC for watermarks
- Handle late-arriving data
- Log processed record counts
- Store watermark safely
- Design idempotent pipelines

12. Key Takeaways

- Incremental loads are standard in production
- Watermarks track pipeline progress

- Timestamps are preferred over IDs
- Handle updates and late data
- Always design for idempotency

Interview Questions

1. What is an incremental load?
2. Difference between full and incremental loads?
3. What is a watermark?
4. How do you handle late-arriving data?
5. What if records are updated after initial load?
6. How do you store pipeline state?

 # Leave the answers in comment